

Teste Técnico Back-end — C# (.NET 8)

Este documento apresenta um **teste técnico de back-end** para candidatos(as) à vaga de **Desenvolvedor(a) Back-end**, com foco em **C# (.NET 8)**. O desafio foi estruturado no formato de **histórias de usuário**, refletindo a forma como os requisitos são trabalhados no dia a dia do time.

O objetivo deste teste é avaliar a capacidade do(a) candidato(a) de compreender necessidades de negócio, traduzi-las em regras técnicas e desenvolver uma **API REST** funcional, organizada e preparada para evolução, considerando cenários reais de **alto volume de dados e alta demanda**.

A proposta **não é avaliar soluções complexas de infraestrutura ou escalabilidade avançada**, mas sim a clareza de raciocínio, a organização do código e o domínio de conceitos fundamentais de back-end, como APIs REST, validações, tratamento de erros e boas práticas de desenvolvimento.

O desafio foi pensado para permitir que desenvolvedores(as) de diferentes níveis consigam demonstrar seu conhecimento. Uma solução funcional e bem estruturada é suficiente, e decisões técnicas adicionais ou explicações sobre possíveis evoluções do sistema serão consideradas como diferencial.

Não é necessário desenvolver interface visual ou implementar autenticação. Caso alguma funcionalidade não seja implementada, o(a) candidato(a) pode descrevê-la no README, explicando a abordagem que adotaria.

História de Usuário — Gerenciamento de Tarefas (Alta Demanda)

Como

Como **operador das áreas de Operações, Suporte ou Logística,**

Eu gostaria de

Criar, consultar, atualizar e remover tarefas por meio de uma **API centralizada,**

Para que

Eu consiga organizar e acompanhar o grande volume de atividades operacionais do dia a dia, mesmo em cenários de alta demanda e muitos acessos simultâneos.

Critérios de Aceitação

Criação de tarefas

Quando eu enviar uma requisição para criação de tarefa,

Então o sistema deve criar a tarefa com **status inicial Pending.**

Quando eu não informar o título da tarefa,

Então o sistema deve **bloquear a criação** e retornar uma mensagem de validação.

Quando a tarefa for criada com sucesso,

Então o sistema deve gerar automaticamente as datas de **criação e atualização.**

Consulta de tarefas (alto volume)

Quando eu consultar a lista de tarefas,

Então o sistema deve retornar os dados de forma **paginada**, evitando o retorno de grandes volumes de uma única vez.

Quando eu informar filtros (ex.: status),

Então o sistema deve retornar apenas as tarefas que atendem aos critérios informados.

Quando não existirem tarefas cadastradas,

Então o sistema deve retornar uma **lista vazia**, sem erro.

Consulta de tarefa por identificador

Quando eu consultar uma tarefa informando um identificador válido,

Então o sistema deve retornar os dados completos da tarefa.

Quando eu consultar uma tarefa com identificador inexistente,

Então o sistema deve retornar uma mensagem informando que a tarefa não foi encontrada.

Atualização de tarefas

Quando eu atualizar o título ou descrição de uma tarefa existente,

Então o sistema deve persistir as alterações e atualizar a data de **última modificação**.

Quando eu tentar atualizar o status da tarefa,

Então o sistema deve respeitar o seguinte fluxo:

Pending → InProgress → Done

Quando eu tentar pular ou regredir o status,

Então o sistema deve **bloquear a operação** e retornar uma mensagem de regra de negócio.

Quando a tarefa estiver com status Done,

Então o sistema não deve permitir alterações de status.

Remoção de tarefas

Quando eu solicitar a remoção de uma tarefa existente,

Então o sistema deve remover a tarefa com sucesso.

Quando eu tentar remover uma tarefa inexistente,

Então o sistema deve retornar uma mensagem informando que a tarefa não foi encontrada.

Regras de Negócio

- Toda tarefa deve possuir um título obrigatório.
- Toda tarefa deve iniciar com status Pending.
- O fluxo de status é linear e não permite regressão ou salto de etapas.
- Tarefas finalizadas (Done) são consideradas encerradas.

- O sistema deve ser preparado para **grande volume de dados e alta frequência de requisições**.
 - Listagens devem ser paginadas e permitir filtros para reduzir carga.
 - As datas de criação e atualização devem ser geradas exclusivamente pelo sistema.
-

Definição de Pronto (DoD)

- API implementada em **ASP.NET Core (.NET 8)**.
 - Endpoints de criação, consulta, atualização e remoção funcionando.
 - Regras de negócio respeitadas em todos os cenários.
 - Paginação implementada na listagem de tarefas.
 - Mensagens de erro retornadas de forma clara e consistente.
 - Projeto documentado com instruções de execução no README.
-

Cenários de Teste

- Criação de tarefa sem título → sistema bloqueia e retorna mensagem de validação.
- Criação de tarefa válida → sistema cria tarefa com status Pending.
- Consulta de tarefas com grande volume → sistema retorna dados paginados.
- Consulta por status → sistema retorna apenas tarefas do status informado.
- Atualização de status seguindo o fluxo permitido → sistema aceita.
- Tentativa de regressão ou salto de status → sistema bloqueia a operação.
- Remoção de tarefa existente → sistema remove com sucesso.
- Remoção de tarefa inexistente → sistema retorna mensagem adequada.